

Content Brain

The complete skill, human-readable. The orchestrator and all nine phases, exactly as the skill runs them, from preflight through Voice DNA to handoff.

Contents

SKILL.md	04-ingest.md
00-preflight.md	05-transcribe.md
01-interview.md	06-tag.md
02-source-discovery.md	07-voice-dna.md
03-vault-setup.md	08-handoff.md
	connector-playbook.md

SKILL.md

/content-brain

Build a person their own **content brain**: an Obsidian vault holding every piece of content they have ever made, transcribed, organized, tagged to their own frameworks, and topped with a full Voice DNA set. One command, an interview, and a guided pipeline that does the technical work for them.

This skill builds the **foundation**. It stops before any content-generation engine. It never writes posts, cuts clips, schedules, or publishes. When it finishes, the person owns a complete, searchable brain and is free to build a content system on top of it.

Who is running this

The **person themselves**, on **their own Mac**, likely non-technical. You are their guide and their hands. You run the installs, the downloads, and the transcription for them, and you explain each step in plain, jargon-free English. You never assume they know what a terminal, an API, or a transcript is. Ask, confirm, then do.

Absolute rules

1. **Consent before every access.** Never scan their disk, download from an account, or read a folder until they have said yes to that specific thing. Show them exactly what you are about to touch.
2. **Nothing leaves their machine** except through connectors they explicitly authorize (e.g. their own YouTube, their own Drive). Their content is theirs and stays local.

3. **Never invent their content.** Only ingest what actually exists. Do not fabricate transcripts, notes, or voice traits. If something cannot be found or fetched, say so and move on.
4. **Plain English always.** Explain what each tool does before you run it. No unexplained commands.
5. **Resumable.** This can take a while and can be stopped and restarted. Track progress and always offer to continue where they left off.
6. **This is the whole product.** Do not build, describe, or hint at the content-generation engine that could sit on top. The handoff points at "you can build your own" in generic terms only.

The pipeline

Run these phases in order. **Load the matching reference file only when you reach that phase**, do not read them all up front. Each reference file is in `references/` next to this file.

#	Phase	Reference file
0	Preflight, install/verify tools	<code>references/00-preflight.md</code>
1	Interview, who they are, their pillars, where content lives	<code>references/01-interview.md</code>
2	Source discovery + connector resolution	<code>references/02-source-discovery.md</code>
3	Vault setup, Obsidian, structured like ours	<code>references/03-vault-setup.md</code>
4	Ingest, pull everything in	<code>references/04-ingest.md</code>
5	Transcribe, audio + video to text	<code>references/05-transcribe.md</code>
6	Tag, content against their frameworks	<code>references/06-tag.md</code>
7	Voice DNA, the four-part set	<code>references/07-voice-dna.md</code>
8	Handoff, what they have, what's next	<code>references/08-handoff.md</code>

The connector playbook (`references/connector-playbook.md`) is a lookup table used inside phase 2 and phase 4, read it when resolving a source.

Bundled scripts

Heavy lifting lives in `scripts/`, referenced with the plugin root path. Always call them as:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/<name>.sh <args>
```

- `preflight.sh`, check and (with consent) install Homebrew, ffmpeg, yt-dlp, Whisper; check for Obsidian.
- `vault_init.sh <vault-path>`, scaffold the vault folder structure and starter notes. Idempotent.
- `ingest_youtube.sh <url> <dest-dir>`, download a channel/playlist/video (audio + metadata + captions) with yt-dlp.
- `ingest_rss.sh <feed-url> <dest-dir>`, pull an RSS/Atom feed (podcast, Substack, Beehiiv) into per-item markdown.
- `transcribe.sh <file-or-dir> <dest-dir>`, transcribe audio/video with Whisper, or lift existing captions.
- `local_scan.sh <root-dir>`, list candidate content files under a directory. Lists only; copies nothing.

Every script prints what it is about to do and is safe to re-run.

Progress + resume

Store progress in the member's vault at `.content-brain/state.json` . On start:

1. If a vault path is already known and `.content-brain/state.json` exists, read it, summarize what is done, and offer to continue from the first unfinished phase.
2. Otherwise start at phase 0.

After each phase completes, update `state.json` with the phase name, a timestamp, and a short result summary (e.g. counts ingested). Keep it small and human-readable.

State file shape:

```
{
  "vault_path": "/Users/<name>/Documents/<Vault>",
  "started": "<ISO date>",
  "phases": {
    "preflight": { "done": true, "at": "<ISO>", "notes": "ffmpeg, yt-dlp, whisper ok" },
    "interview": { "done": true, "at": "<ISO>", "notes": "5 pillars captured" },
    "...": {}
  }
}
```

Tone

Warm, concrete, and calm. This person is trusting you with their life's work and their own machine. Move at their pace, confirm before anything irreversible, and celebrate the milestones ("that's 214 videos transcribed and searchable now"). When a step will take a while, tell them roughly how long and that they can walk away.

Start

Greet them, explain in two sentences what you are about to build together, then begin **phase 0** by loading `references/00-preflight.md` .

```
references/00-preflight.md
```

Phase 0: Preflight

Purpose: get the member's Mac ready with the free tools we need, and decide where their vault will live.

Work through these steps in order. Say things in your own warm words, the quoted lines are examples of tone, not a script to read verbatim. Always ask before you install or touch anything.

1. Greet and set expectations

Introduce what you're building together in two plain sentences, then tell them what this first step is.

Say something like:

```
"We're going to build you a 'content brain', one tidy place on your Mac that holds everything you've ever made (posts, videos, podcasts), all searchable and organized, plus a profile of how you sound. I'll do the technical parts and explain each one as we go."
```

"First I need to get a few small, free tools onto your Mac. This part takes about 10 to 20 minutes, most of it just waiting while things download, you can grab a coffee once it's running."

Reassure them: everything stays on their own machine, and you'll ask before doing anything.

2. Decide where the vault will live

This matters, every later step reads and writes to this folder, so pin it down now.

Ask where they'd like their content brain to live, and offer a sensible default:

"Where should your content brain live on your Mac? If you're not sure, a good default is a folder in your Documents. What's your name or brand called? I'll put it at `~/Documents/<that> Content Brain`."

Once they choose (or accept the default), confirm the full path back to them in plain terms ("So it'll live at ... , sound good?").

Then create the tracking folder and an initial state file. Substitute the chosen path for `<vault>`:

- Run: `mkdir -p "<vault>/content-brain"`
- Write `<vault>/content-brain/state.json` with this starting content (fill in the real path and the current time in ISO 8601):

```
{
  "vault_path": "<vault>",
  "started": "<ISO timestamp now>",
  "phases": {}
}
```

Hold onto `vault_path`, every later phase uses it. Note for your own tracking: the actual vault folders (Home.md, wiki/, etc.) do NOT exist yet; Phase 3 builds them. Right now only the hidden `content-brain/` folder exists, and that's correct.

3. Check what's already installed (with consent)

Tell them you'd like to look at what's already on their Mac, a read-only check that installs nothing yet.

Say:

"Let me take a quick look at what your Mac already has. This just checks, it won't install or change anything. Okay to run it?"

After they say yes, run:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/preflight.sh
```

It prints one line per tool in this shape:

```
STATUS<TAB>tool<TAB>state<TAB>detail
```

where `state` is one of `ok`, `missing`, `installed`, or `failed`. Parse each line. The tools it reports are: `brew`, `ffmpeg`, `yt-dlp`, `python3`, `whisper`, `obsidian`.

4. Report what you found, in plain English

Tell them which tools are already present and which are missing. When you name a tool, give it one friendly line so they know why it's there:

- **Homebrew**, the free installer that fetches the other tools for you.
- **ffmpeg**, handles audio and video files behind the scenes.
- **yt-dlp**, downloads your videos so we can bring them in.
- **python3**, a small helper that Whisper needs to run.
- **Whisper**, turns speech into text. It's a large one-time download (about 2 to 3 GB) and it's only needed if some of your content has no captions.
- **Obsidian**, the free app that holds your brain and lets you browse it.

Keep it upbeat: "Good news, you already have most of this. We just need two more."

5. Install anything missing (with consent, Whisper optional)

If nothing is missing, skip to step 6. Otherwise, list exactly what you'd like to install and ask permission first:

"You're missing ffmpeg and yt-dlp. These are free and safe, and Homebrew will fetch them. This is the part that takes a few minutes. Want me to go ahead?"

If Homebrew itself needs installing, warn them it may ask for their Mac password, that's normal and it's their own computer asking.

After they say yes, run:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/preflight.sh --install
```

Offer to defer Whisper. Whisper is the heavy one. If it's missing, give them the choice:

"One of these, Whisper, is a big download (2 to 3 GB) and it's only needed later, for any videos or audio that don't already have captions. Lots of content comes with captions, so we can skip it for now and grab it later only if we actually need it. Install it now, or hold off?"

If they'd rather wait, note that Whisper is deferred to Phase 5 and don't install it now.

6. Re-check and handle any failures

Run the check again to confirm the installs took:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/preflight.sh
```

Report the updated results. Then:

- **If everything now shows `ok` or `installed`**: tell them they're all set and move on.
- **If a tool shows `failed`**: stay calm and don't treat it as the end of the road. The `detail` field on that STATUS line is a one-line manual command (e.g. `brew install ffmpeg`). Share it plainly:

"One tool didn't install cleanly. You can finish it yourself by pasting this one line into your Terminal: `brew install ffmpeg`. Want to try that, or shall I?"

- **Offer to continue without it where possible.** If Whisper is the one that failed or was deferred, reassure them we can keep going and only come back to transcription later:

"No problem, we can build everything else and only revisit transcription once your content's in. Captions will cover a lot on their own."

Only the missing tool's own feature is affected (e.g. no Whisper just means transcription waits). Don't block the whole pipeline on one optional tool.

When this phase is done

1. Update `<vault>/content-brain/state.json`: set `phases.preflight` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<short summary, e.g. 'ffmpeg, yt-dlp, obsidian ok; whisper deferred'>" }`.
2. Load `references/01-interview.md` and begin Phase 1.

`references/01-interview.md`

Phase 1: Interview

Purpose: understand the person, capture their frameworks and pillars (these become the tag vocabulary later), decide whether their personal voice differs from their brand voice, and map exactly where all their content lives.

How to run this whole phase:

- **One question at a time.** Never stack questions or hand them a form. Ask, wait, listen.
- **Keep questions concrete and easy.** "Which platforms do you post on?" beats "describe your content strategy."
- **Reflect back what you heard** after each section, so they can correct you: "So your audience is early-stage founders, did I get that right?"
- Move at their pace. This is a conversation, not an intake.

Work through sections A, B, C, D in order.

A. Who they are

Get name, business, and audience, one question at a time.

Ask, in turn:

"First, what should I call you?"

then:

"In a sentence or two, what does your business do?"

then:

"And who's it for, who's your audience?"

If they have a website, offer to read it so they don't have to type much:

"Do you have a website? If you share the link, I can read it and pull the basics myself, you just tell me what I got wrong."

If they give a URL, use WebFetch to read it, then confirm what you learned back to them in plain terms:

"Okay, from your site it looks like you help [X] do [Y]. Accurate?"

Never state anything the site didn't actually say. If the fetch fails or is thin, just ask them directly.

B. Their content pillars and frameworks

Capture the 3 to 7 big themes or named frameworks they're known for. This list is important, later phases tag every piece of content against it, so get it right.

Explain simply what you're after:

"Most people who make content come back to a handful of core themes, or have named frameworks they teach. What are the big ideas you keep returning to? Anywhere from three to seven."

If you read their website in section A, draft a candidate list first and have them confirm or edit it, rather than making them start from a blank page:

"From your site, it looks like your main themes might be: [theme 1], [theme 2], [theme 3]. Which of these feel right, and what am I missing?"

Refine until they're happy. Read the final list back and confirm it's the one to lock in.

C. Voice check

Decide whether the way they talk as a PERSON differs from their BRAND voice. This one answer decides whether Phase 7 builds a separate Personal Voice DNA, so ask it clearly.

Say:

"Quick one about voice. Some people sound exactly the same whether it's the brand talking or them personally. Others have a polished brand voice and a looser, more casual way they talk as themselves. Which are you, same voice, or noticeably different?"

If they're unsure, give an example to help: "For instance, is your LinkedIn more buttoned-up than how you'd text a friend about the same idea?"

Record their answer as a simple yes/no: does the personal voice differ from the brand voice?

D. Content inventory

Map where all their content lives and what type it is. Walk them through it **one channel at a time**, don't dump the whole list on them. Capture handles, URLs, and rough locations. **Do NOT access, download, or open anything yet**, this is only mapping. Access comes in Phase 2, with consent.

Go type by type. For each, ask whether they have it and, if so, where:

Text

"Let's map your writing. Do you have a blog or write articles anywhere?"

Then, one at a time as relevant: a newsletter (Substack, Beehiiv, or another), LinkedIn, X/Twitter, any published books, any PDFs or guides. Capture the handle or URL for each.

Video

"Now video. Are you on YouTube?"

Then, as relevant: online courses, webinars, Vimeo, recorded calls or Zoom sessions. Capture channel URLs or where the files sit.

Audio / podcast

"Do you have a podcast?"

If yes, ask which host it's on (Apple, Spotify, Buzzsprout, Libsyn, etc.) and whether they happen to know the RSS feed URL, reassure them it's fine if they don't, you can find it later.

On their computer Ask plainly:

"Last one, is there content just sitting on your computer? Folders of videos, documents, recordings, anything like that?"

If yes, ask roughly where (e.g. "a Videos folder on my desktop"), a rough location is enough for now. Don't open or scan it.

Reflect the whole inventory back once you've walked through all four:

"Here's what I've got for where your content lives: [list]. Anything I missed?"

Persist everything captured

Write everything from this phase into `<vault>/content-brain/state.json` (the `vault_path` was set in Phase 0).

Add an `interview` object at the top level alongside `vault_path`, `started`, and `phases`. Note: only the `content-brain/` folder exists right now, the real vault folders arrive in Phase 3, so writing here is correct.

Shape:


```

{
  "interview": {
    "profile": {
      "name": "<their name>",
      "business": "<one-line what the business does>",
      "audience": "<who it's for>",
      "website": "<url or null>"
    },
    "pillars": ["<pillar 1>", "<pillar 2>", "..."],
    "personal_voice_differs": true,
    "inventory": [
      { "type": "text", "where": "Substack", "detail": "https://... / handle" },
      { "type": "video", "where": "YouTube", "detail": "channel URL" },
      { "type": "audio", "where": "podcast", "detail": "host + RSS if known" },
      { "type": "local", "where": "Mac", "detail": "rough folder location" }
    ]
  }
}

```

Fill in real values. Use `null` for anything they don't have. Keep it small and human-readable.

When this phase is done

1. Update `<vault>/content-brain/state.json`: set `phases.interview` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<short summary, e.g. '5 pillars, personal voice differs, 4 content sources'>" }`.
2. Load `references/02-source-discovery.md` and begin Phase 2.

`references/02-source-discovery.md`

Phase 2: Source Discovery + Connector Resolution

Purpose: turn the interview's list of where their content lives into a concrete, approved ingestion plan. In this phase you decide HOW each source will be pulled in and get their sign-off. You do not ingest anything yet. The only actions allowed here are light, read-only checks.

Say something like: "Now I'll figure out the best way to pull each of these in, and do a few quick, harmless checks. I won't download or copy anything until you've seen the plan and said go."

Step 1: Read the inventory

Open the vault's progress file at `<vault>/content-brain/state.json` and read `interview.inventory`. That is the list of content homes the member gave you (each with a type and a location: a URL, an app, a folder). Work through it item by item.

Step 2: Resolve each source

For every inventory item, consult `references/connector-playbook.md` and pick the path. Then do a quick, SAFE verification only, enough to know the path will work. Copy nothing.

Safe checks you may run:

- **YouTube:** confirm the handle or playlist resolves with a dry listing, e.g. `yt-dlp --flat-playlist --skip-download <url>` (lists titles, downloads nothing). Note the rough count.
- **RSS / newsletter / podcast:** check the feed URL exists (a WebFetch or a `curl -fsI`). Confirm it looks like a feed before committing to it.
- **Blog:** check for `/sitemap.xml` or a feed so you know how many articles exist.
- **Google Drive / Gmail:** confirm the connector is actually connected (discover the tool via ToolSearch). If it is not, that becomes a member-action item, not a check you force.
- **Local folder:** only with explicit consent, run `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder>` so they can SEE what is there before deciding. It lists and copies nothing. See Step 4.

Do not download, transcribe, or read account content in this phase. If a check is not clearly harmless, skip it and note the item as "will confirm at ingest."

Step 3: Flag member-action items

Some paths need the member to do something that takes time or a decision. Pull these out and list them clearly and early so they can start now while you keep working:

- **Exports that take time:** X/Twitter archive (can be a day+), LinkedIn data copy, Notion export, course/Zoom downloads.
- **A connector to link:** Google Drive or Gmail not yet connected.
- **A key or token to supply:** e.g. Notion API. Remind them the key goes into their own environment, never the vault or a script, and you will tell them exactly where to paste it.

Present these as a short "kick these off now" checklist. Example:

"A few of these need a quick move from you before I can pull them in:

1. X/Twitter: request your archive now (Settings > Download an archive) so it's ready by the time we get there.
2. Google Drive: connect it as a connector when you have a sec, or export those docs to a folder. Everything else I can handle. Want to start these while I map out the rest?"

Step 4: Local folders need explicit consent

Before you run even the read-only `local_scan.sh` on a folder, ask for a clear yes for that exact folder:

"You mentioned your writing is in `~/Documents/Writing`. Can I do a read-only look inside just to list what's there? I won't open, move, or copy a single file, just show you the list."

After they agree, run `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder>`, then show them the result and let them decide what is in and what is out. Never scan a folder they did not name.

Step 5: Assemble the plan and get approval

Build a plain table of every source. Keep it skimmable:

Source	Type	Path	Rough volume	Prerequisite
@theirchannel	YouTube	ingest_youtube.sh	~180 videos	none

Source	Type	Path	Rough volume	Prerequisite
Their podcast	Podcast	RSS + transcribe	~60 episodes	find feed URL
Substack	Newsletter	RSS (/feed)	~40 posts	none
~/Documents/Writing	Local text	local_scan.sh	220 files found	consent given
X archive	Social	archive import	unknown	member requesting export

Present it and ask for approval in plain terms:

"Here's the full plan for what I'd pull in and how. Look it over. Anything here you'd rather leave out? Anything I missed? Nothing gets touched until you say go."

Let them drop anything they do not want. Adjust the table to match their choices.

Step 6: Save the approved plan

Write the approved plan into `state.json` under `ingestion_plan` (an array of the sources with their path, destination subfolder, rough volume, and any prerequisite/member-action status). Keep it small and human-readable so a later resume can pick it straight up.

When done

1. Mark source discovery complete in `state.json` :

```
"source_discovery": { "done": true, "at": "<ISO>", "notes": "N sources approved, M awaiting member export" }
```

2. Load `references/03-vault-setup.md` and begin Phase 3.

Tell them what is next: "Plan's locked in. Next I'll set up your Obsidian vault, the home everything lands in, and then start pulling your content in."

`references/03-vault-setup.md`

Phase 3: Vault setup

Purpose: build their Obsidian vault, structured exactly like ours, and open it so they can watch their brain take shape.

By now the interview (Phase 1) and source discovery (Phase 2) are done, and `state.json` holds their profile, their pillars, and the approved ingestion plan. This phase creates the (still empty) vault and seeds it with their frameworks and brand. Nothing is ingested yet, that's Phase 4.

1. Read the vault path

Open `<vault>/content-brain/state.json` and read `vault_path`, you set this back in Phase 0. Every command below writes inside that folder. If it's missing, something went wrong earlier; go back and re-confirm the path rather than guessing one.

2. Explain Obsidian in one friendly line

Before you build anything, tell them what they're about to see. One plain sentence:

"Obsidian is a free app where every note is just a plain-text file sitting on your Mac. The notes link to each other to form one connected map of your work, and because it's all plain text it's yours forever, no cloud, no lock-in."

3. Scaffold the vault

Run the init script. It creates the folders and starter notes, and it's safe to run more than once (it keeps anything that already exists):

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/vault_init.sh "<vault_path>"
```

It prints one `wrote` or `kept` line per file and finishes with:

```
VAULT_READY<TAB><vault_path>
```

Tell them what it made, in plain terms: a Home note, an index (the map of everything), a running log, and a set of `wiki/` folders, `sources/` for their content, `voice/` for their Voice DNA, `concepts/` for their frameworks, plus entities, systems, builds, and prompts.

4. Seed their frameworks (concepts)

Their pillars are the tags everything will hang on later, so create one note per pillar now. Read `interview.pillars` from `state.json`. For each pillar, draft a one-line description in their language, read it back, and let them confirm or fix it before you write:

"Your first pillar is 'Owner Financing.' I'd describe it as: 'Buying and selling businesses using seller-held notes instead of bank loans.' Does that capture it, or want to tweak the wording?"

Then write `wiki/concepts/<Pillar>.md` with short frontmatter and the confirmed line:

```
---
title: Owner Financing
type: concept
tags: []
---

Buying and selling businesses using seller-held notes instead of bank loans.
```

Use the pillar's real name for both the `title` and the filename. Repeat for every pillar. Don't invent pillars they didn't name.

5. Create their brand entity

Read `interview.profile` and write one note for their business in `wiki/entities/`. Capture the name, what it does, and who it's for:

```
---
title: <Business Name>
type: entity
tags: []
---

**What it does:** <one line from their profile>
**Audience:** <who it's for>
```

Keep it to what they actually told you, don't add positioning or claims they didn't say.

6. Make sure Obsidian is installed

This is the moment Obsidian is actually needed, so confirm it's really here before you rely on it, even though Phase 0 should have installed it. Check both locations:

```
ls -d /Applications/Obsidian.app "$HOME/Applications/Obsidian.app" 2>/dev/null
```

If neither path exists, Obsidian isn't installed yet (the Phase 0 install may have been skipped or failed). Install it now, with consent, before going further:

- **Preferred:** `brew install --cask obsidian`
- **If that fails or Homebrew isn't available:** point them to the direct download and wait for them:

"Obsidian is the free app your brain lives in. Grab it from <https://obsidian.md>, open the download, drag Obsidian to your Applications folder, then tell me and I'll open your vault for you."

Do not move on until Obsidian is present.

7. Open the vault

Show them the result. Try:

```
open -a Obsidian "<vault_path>"
```

If Obsidian opens but doesn't show the vault, tell them how to point it there themselves: open Obsidian, choose "Open folder as vault," and pick the folder at `<vault_path>`. Once it's open, confirm it with them and walk their eyes to the `wiki/` folders in the left sidebar, especially `concepts/`, where their pillars now sit, and `sources/`, which fills up next.

When this phase is done

1. Update `state.json`: set `phases.vault_setup` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<short summary, e.g. 'vault scaffolded; 5 pillars + brand entity seeded'>" }`.
2. Load `references/04-ingest.md` and begin Phase 4.

`references/04-ingest.md`

Phase 4: Ingest

Purpose: run the ingestion plan you agreed on in Phase 2 and pull everything into the vault as organized, linked markdown plus media, one note per item, ready to be transcribed (Phase 5) and tagged (Phase 6).

Read the approved plan from `state.json` (the sources and connectors you resolved in Phase 2) and work it one source at a time.

Before you start

- **Set expectations.** This is the long phase. Give them a rough sense of time based on how much they have, and tell them they can walk away:

"You've got about 200 YouTube videos and two podcast feeds. The downloads will take a while, maybe an hour, mostly unattended. Feel free to grab a coffee, I'll keep going and tell you where I land."

- **Consent is already captured.** You only touch what they approved in Phase 2. In particular, never copy anything off their local disk unless they said yes to that specific folder earlier. If a source wasn't approved, skip it.
- **Show progress.** After each source, tell them the running count:

"That's 214 YouTube notes and 63 podcast episodes in so far."

Principles

- Work **source by source**, finish one before starting the next.
- Every ingested item becomes **one markdown note** with frontmatter. Leave a `tags: []` field empty; Phase 6 fills it against their pillars.
- **Dedupe** by source URL, file hash, or title + date. Never write the same item twice.
- **Never invent content.** Only write notes for things that actually downloaded or exist. If a fetch fails, say so and move on, don't fabricate a note.
- **Keep a per-item record** so a re-run skips what's already done (see "Track progress").

YouTube

Run:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/ingest_youtube.sh <url> "<vault>/wiki/sources/youtube"
```

It downloads audio (`.m4a`), metadata (`.info.json`), captions (`.vtt` / `.srt`), and a thumbnail for each video, skipping anything already fetched. It ends with `YT_DONE<TAB><dest>`.

Then, for **each** `.info.json`, write a note at `wiki/sources/youtube/<date>-<slug>.md`:

```

---
title: <video title>
type: youtube
source: <video url>
date: <YYYY-MM-DD upload date>
channel: <uploader>
duration: <hh:mm:ss>
video_id: <id>
tags: []
---

Downloaded files:
- Audio: [[<relative path to the .m4a>]]
- Captions: [[<relative path to the .vtt/.srt, if present>]]

```

Pull every field from the `.info.json` (title, uploader, upload_date, duration, id), don't guess them. Link to the actual downloaded audio and caption files by their real paths (the script saves them under a per-channel subfolder). Transcription happens in Phase 5, so there's no transcript yet.

Podcast / RSS

Run:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/ingest_rss.sh <feed-url> "<vault>/wiki/sources/podcast"
```

It saves the raw feed to `feed.xml` and prints item links and audio enclosure URLs as hints. **Read `feed.xml` yourself as the authoritative source**, the printed greps are only a preview, not a full parser. It ends with `RSS_DONE<TAB><raw>`.

Write one note per episode at `wiki/sources/podcast/<date>-<slug>.md`:

```

---
title: <episode title>
type: podcast
source: <episode page url>
date: <YYYY-MM-DD>
audio_url: <enclosure url>
tags: []
---

<episode description / show notes from the feed>

```

Keep the `audio_url`, Phase 5 uses it to fetch and transcribe the audio.

Newsletter / blog articles

Two paths, depending on what Phase 2 resolved:

- **RSS:** run `ingest_rss.sh <feed-url> "<vault>/wiki/sources/articles"` and turn each item into a note.

- **Individual URLs:** fetch each with WebFetch (load it via ToolSearch first if it isn't available yet).

Write one clean note per article at `wiki/sources/articles/<date>-<slug>.md` :

```
---
title: <article title>
type: article
source: <url>
date: <YYYY-MM-DD>
tags: []
---

<the full article text>
```

Preserve the full text, never truncate the author's own words. This corpus is what the Written Voice (Phase 7) is mined from, so completeness matters.

Google Drive / Docs

If a Drive connector was approved, load the Drive MCP tools via ToolSearch (e.g.

`mcp__claude_ai_Google_Drive__search_files` , `mcp__claude_ai_Google_Drive__read_file_content`). Find the docs they pointed you at, read each one, and write it as a markdown note in the right `sources/` subfolder (`articles/` , `books/` , or `transcripts/` as fits):

```
---
title: <doc title>
type: <article|book|transcript>
source: <drive file id>
date: <YYYY-MM-DD>
tags: []
---

<the document text>
```

Local files (only what they consented to)

Only for folders they explicitly approved in Phase 2. For each approved file:

- **Copy** the original into `wiki/sources/assets/` so the vault is self-contained, **except** very large media (big video/audio files), which you should leave in place and link by absolute path to avoid bloating the vault. Use your judgment on the size cutoff, and tell them which you did:

"I copied your documents into the vault and linked your 4 GB video files where they already sit."

- **Extract the text** so it's searchable and mineable later:
 - `.docx` , `.doc` , `.rtf` → `textutil -convert txt "<file>" -output "<out>.txt"`
 - `.pdf` → `pdftotext "<file>" "<out>.txt"` (fall back to `textutil -convert txt` if `pdftotext` isn't installed)
 - `.epub` → `ebook-convert "<file>" "<out>.txt"` (Calibre; skip if not installed)
- **Write one note per document** with the extracted text and frontmatter:


```
---
title: <file name or doc title>
type: <book|article|transcript|asset>
source: <absolute path to the original>
date: <file date if known>
tags: []
---

<the extracted text>
```

Dedupe

Before writing any note, check you haven't ingested the item already, match on source URL, file hash, or title + date. Skip duplicates silently rather than writing a second copy.

Track progress

After each source finishes:

1. Update the counts in `Home.md` and `index.md` (e.g. under `## Sources`, list `youtube, 214`, `podcast, 63`).
2. Append a dated line to `log.md` (e.g. `2026-07-14, ingested 214 YouTube notes from @channel`).
3. Record what's done in `state.json` (e.g. per-source processed ids/hashes, or a simple done-list) so a re-run skips finished items and only picks up what's new.

When this phase is done

1. Update `state.json`: set `phases.ingest` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<counts by type, e.g. 'youtube 214, podcast 63, articles 40, local 12'>" }`.

2. Give them the tally in plain English:

"Everything's in: 214 videos, 63 podcast episodes, 40 articles, and 12 documents, all as linked notes in your vault."

3. Load `references/05-transcribe.md` and begin Phase 5.

`references/05-transcribe.md`

Phase 5: Transcribe

Purpose: turn all their ingested audio and video into searchable text, so every spoken word becomes something they can find, tag, and mine for voice later.

Work through these steps in order. Say things in your own warm words, the quoted lines are examples of tone, not a script to read verbatim.

1. Sort out Whisper (only if you deferred it)

If you deferred the Whisper install back in Phase 0, check now whether any of their media actually lacks captions. A lot of content, especially YouTube, comes with captions already, and those are instant and free.

- If everything has captions, you don't need Whisper at all. Say so and move on to step 2.
- If some media has no captions, offer to install Whisper now, and be honest that it's a large one-time download:

"A few of your videos don't have captions, so to read those I'd need Whisper, the tool that turns speech into text. It's a big one-time download (2 to 3 GB). The captioned stuff I can do without it. Want me to grab Whisper now so we get everything, or skip it and just do the captioned ones for today?"

If they say yes, install it with `"${CLAUDE_PLUGIN_ROOT}"/scripts/preflight.sh --install`, then come back here. If they'd rather skip, that's fine, the captioned items still get done.

2. Run the transcriber over each media folder

For every folder that holds audio or video, run `transcribe.sh`. Point it at the source folder and send the output to the shared `transcripts` folder. For example:

```
"${CLAUDE_PLUGIN_ROOT}"/scripts/transcribe.sh "<vault>/wiki/sources/youtube" "<vault>/wiki/sources/transcr
```

Do the same for `podcast/` and any other folder with media in it.

How it behaves:

- It **prefers existing captions**, those convert to text instantly and for free.
- It only runs **Whisper on items that have no captions**.
- The model argument is optional. Default `small` is a good balance of speed and accuracy for spoken word. `tiny` and `base` are faster but rougher; `medium` and `large` are more accurate but slower and heavier. Mention these if they ask, but `small` is a fine default.

3. Set expectations honestly

Big libraries can take a while. Tell them the truth before you start so nobody's watching a spinner:

"Anything with captions is a second or two each. Anything I have to transcribe with Whisper runs somewhere around real-time, a rough fraction of the clip's length depending on the model and how fast your Mac is, so a big back catalog can take a while. Good news: you can walk away. It saves as it goes, skips anything already done, and if we stop and restart it just picks up where it left off."

4. Write a transcript note and link it back to the source

For each `.txt` the script produces in `wiki/sources/transcripts/`:

- Create or attach a transcript note in `wiki/sources/transcripts/` holding the **full transcript text, never truncate it**. Preserve every word.
- **Link it from the matching source note** so they travel together: add a `transcript:` link to the source note (in its frontmatter, or in the body) pointing at the transcript note, e.g. `transcript: "[[<source-stem>transcript]]"`.

This two-way connection is what lets them jump from a video straight to its words and back.

5. Report the counts

When the run finishes, give them a plain-English tally:

"Done. That's 118 pulled straight from captions, 46 transcribed with Whisper, and 3 skipped. Here are the 3 that didn't go through, so we can retry them: [list]."

List any failures by name so they can retry. A failure is not the end, the run is resumable, so retrying just re-attempts the ones that didn't finish.

When this phase is done

1. Update `<vault>/content-brain/state.json` : set `phases.transcribe` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<counts, e.g. '118 from captions, 46 whispered, 3 failed'>" }`.
2. Load `references/06-tag.md` and begin Phase 6.

`references/06-tag.md`

Phase 6: Tag

Purpose: connect every piece of content to the member's own frameworks, so the vault becomes one navigable graph instead of a pile of files. When this is done, they can click a pillar and see everything they've ever made about it.

This phase touches a lot of files. Work in batches, show progress as you go, and skip anything already tagged, it's resumable.

1. Load their pillars and concept notes

Read the member's pillars from `state.json` (`interview.pillars`), and open their concept notes in `wiki/concepts/`. These pillars are the tag vocabulary, the whole point is to connect content back to *their* frameworks, not any generic scheme.

2. Tag every source and transcript note

Go through every note in `wiki/sources/` (including the transcript notes). For each one:

- **Read it** so you know what it's actually about.
- Decide which **1 to 3 pillars** it genuinely belongs to. Add them as tags in the note's frontmatter: `tags: [pillar-a, pillar-b]`.
- Add a **few free keywords** drawn from the actual content, real topics, names, and terms that appear in the piece.
- Add a **wikilink** from the note to each relevant concept note, e.g. `[[<Pillar>]]`, so Obsidian's graph view draws the connection.

Never assign a pillar the content doesn't actually support. If a piece only fits one pillar, give it one. If it fits none cleanly, leave it untagged rather than forcing it. Honest tags keep the graph trustworthy.

3. Mark notable moments in long transcripts (optional)

For long transcripts, you may add a short list of **3 to 6 notable moments**, a rough timestamp and a one-line description each, but only where the transcript clearly supports it. This makes the archive searchable at a glance ("where did I talk about pricing?").

Keep it honest and short. **Do not fabricate** moments, quotes, or timestamps. If the transcript doesn't clearly show it, don't list it.

4. Update the concept notes

For each concept note in `wiki/concepts/`, add a short summary or a backlinks-style list of what content now supports it, which videos, articles, and episodes touch that pillar. Obsidian shows backlinks automatically too, but a short human summary at the top makes each pillar note a real hub.

5. Refresh the index and log

- Update `index.md` so it maps their sources by pillar, a reader should be able to scan it and see what they've made under each theme.
- Append a short progress line to `log.md`.

When this phase is done

1. Update `<vault>/content-brain/state.json`: set `phases.tag` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<short summary, e.g. '164 notes tagged across 5 pillars'>" }`.
2. Load `references/07-voice-dna.md` and begin Phase 7.

`references/07-voice-dna.md`

Phase 7: Voice DNA

Purpose: distill how this person actually sounds into a small set of durable notes, so every future word can be checked against their real voice instead of a generic one. This is the centerpiece of the whole build.

You now have their corpus ingested, transcribed, and tagged. That corpus is the evidence. Your job is to read it, draft their voice on file, and walk them through confirming it. **Never invent a trait, phrase, or example that the corpus does not actually show.** If the evidence is thin somewhere, say so and build only what's supported.

You are producing up to **four** cross-linked notes in `wiki/voice/`:

1. **Brand Voice DNA**, how the business sounds
2. **Personal Voice DNA**, how the person sounds as themselves (only if they told you these differ)
3. **Written Voice**, how they write (mined from their text)
4. **Spoken Voice**, how they talk (mined from their transcripts), a genuinely different instrument

Written and spoken are kept separate on purpose. People do not write the way they talk. The pace, the phrases, the sentence shapes are different, and the whole point is to capture both truthfully.

How this phase works

For each note: **gather the evidence** → **analyze it** → **draft the note** → **walk them through it** → **finalize**. Show them what you found and let them correct it. Their confirmation is what makes the file trustworthy.

Tell them up front, warmly: *"This is the part that makes everything sound like you and not like a robot. I'm going to read back through everything you've made, pull out how you actually write and how you actually talk, and then check it with you."*

Step 1: Written Voice

Gather. Pull the text corpus: everything under `wiki/sources/articles/`, `wiki/sources/books/`, and any extracted documents. Use their real, self-written prose, not transcripts (those are Spoken Voice).

Analyze honestly, from the actual text:

- **Sentence shape**, typical length, how much they vary it, short-punch vs long-winding.
- **Structure**, how they open a piece, how they build, how they close. Do they lead with a story, a claim, a question, a number?
- **Punctuation habits**, describe what they *actually* do (lists, dashes, parentheticals, one-line paragraphs, ALL CAPS for emphasis). Report their habits truthfully; do not impose anyone else's rules on their voice.
- **Transitions and connective tissue**, the words they reach for to move between ideas ("Here's the thing", "But", "So", "Look").
- **Vocabulary and register**, plain vs technical, formal vs casual; a reading-grade feel (roughly, from sentence and word length, don't overstate precision).
- **Rhetorical moves**, the reveal, the contrast, the confession, the direct address, the rule-of-three, whatever recurs.

Draft `wiki/voice/Written Voice.md` with frontmatter `type: voice` and sections for each of the above, each backed by **8-12 real quoted lines pulled verbatim from their writing** (cite the source note with a `[[wikilink]]`). The quotes are the proof, they matter more than your adjectives.

Walk them through it. Show the headline findings and 3-4 example quotes. Ask: *"Does this feel like how you write? Anything I've overstated or missed?"* Fold in their corrections.

If there is little or no written content, say so plainly and note that Written Voice is light for now and will grow as they add writing.

Step 2: Spoken Voice

Gather. Pull the transcripts under `wiki/sources/transcripts/`. For pace, you also need durations of the underlying media.

Compute words per minute (real number, not a guess):

- Word count of a transcript: `wc -w "<transcript>.txt"`.
- Duration of its media: from the YouTube `.info.json` `duration` field, or run `ffprobe -v quiet -show_entries format=duration -of csv=p=0 "<media-file>"` (seconds).
- WPM = words ÷ (seconds ÷ 60). Do this across several representative pieces and report a typical range (e.g. "about 165-180 wpm"). Most conversational speakers land somewhere around 130-190 wpm, but report their real measured number, never an assumption.

Analyze from the transcripts:

- **Signature openers**, how they start talking / start an answer.
- **Recurring phrases and verbal tics**, the expressions that show up again and again (quote the real ones, with counts if you can, e.g. "'ask me how I know' appears 14 times").
- **Filler and rhythm**, their natural filler words, pauses, the cadence of how a spoken sentence runs.
- **How they walk to a point**, do they set up with a number and pay it off? Tell a mini-story then land the lesson? Ask then answer?
- **Energy and register when speaking** vs writing, often looser, faster, more digressive.

Draft `wiki/voice/Spoken Voice.md` (`type: voice`) with the WPM figure up top, then each section backed by **verbatim quoted phrases** from real transcripts ([[wikilink]] the source). Note which phrases are frequent vs one-offs.

Walk them through it. Play back the standout findings, the WPM number and their top signature phrases usually land well. *"These are the phrases you actually reach for out loud, sound right? Any you'd never want in your name?"* Fold in corrections.

If there is no audio/video, say so and skip Spoken Voice for now, noting it can be built later once they have recordings.

Step 3: Brand Voice DNA

This is how the **business** sounds in public. Synthesize from their content *and* the interview profile (`interview.profile` , `interview.pillars` in state.json).

Draft `wiki/voice/Brand Voice DNA.md` (`type: voice`) covering:

- **Positioning**, who they help and what they stand for, in a line.
- **Audience**, who they're talking to.
- **Core messages**, the handful of things the brand says over and over (tie each to a [[pillar]] concept note where relevant).
- **Signature vocabulary**, the words and phrases that are theirs; the words they avoid.
- **Tone**, 3-5 adjectives, each with a real example.
- **Do / Don't**, concrete guardrails drawn from how they actually show up (e.g. "Do lead with a real number. Don't use hype adjectives.").

Keep every claim grounded in real examples from the corpus. **Walk them through it** and let them sharpen the do/don't list, that's the part they'll have the strongest opinions on.

Step 4: Personal Voice DNA (conditional)

Only build this if `interview.personal_voice_differs` is true. If their personal and brand voice are the same, skip it and say so, one honest Brand Voice note is better than two padded ones.

If they differ, draft `wiki/voice/Personal Voice DNA.md` (`type: voice`) describing how they sound as *themselves*, the looser, off-brand, this-is-just-me register, drawn mostly from casual spoken moments and any personal writing. Contrast it explicitly with the brand voice ("Brand is X; personally they're more Y"). Walk them through it.

Step 5: Wire it together

- Cross-link the four notes to each other and link each from `index.md` under the Voice section (the starter `index.md` already lists them).
 - Add a one-line pointer in `Home.md` to the Voice set.
 - Make sure every voice trait that cites the corpus uses a real [[wikilink]] to the source note, so a reader can click straight to the evidence.
-

When this phase is done

Update `<vault>/content-brain/state.json` : set `phases.voice_dna` to `{ "done": true, "at": "<ISO>", "notes": "built: written, spoken, brand[, personal]; wpm ~<n>" }`. Note which of the four you built and which you skipped and why.

Then load `references/08-handoff.md` and begin the final phase.

`references/08-handoff.md`

Phase 8: Handoff

Purpose: show the member everything they now own and leave them a clear map they can use on their own. This is the final phase. When it's done, the run is complete.

Say things in your own warm words, the quoted lines are examples of tone.

1. Write the handoff note

Create `wiki/HANDOFF.md` and link it from `Home.md`. It's their map to what they now have. Include:

- **Counts by content type**, how many videos, articles, podcast episodes, books, etc. (use the real numbers from `state.json` and the vault).
- **Where each type lives**, the folder under `wiki/sources/` for each.
- **That it's all transcribed and tagged** to their own pillars, so it's searchable and connected.
- **Where the Voice DNA set lives**, `wiki/voice/` : Brand Voice DNA, Personal Voice DNA (if it was built), Written Voice, and Spoken Voice.

Keep it plain and skimmable, this is the note they'll come back to.

2. A short "how to use Obsidian" primer

They may be new to Obsidian, so give them a few friendly basics for a non-technical owner:

"A few things to know about your new vault:

- Open it in the Obsidian app any time, everything is right here on your Mac.
- Use **search** (top of the sidebar) to find anything by word or phrase.
- Click the **graph view** to see your whole brain as a web of connected notes.
- Any `[[link]]` or `#tag` is clickable, follow them to jump between related pieces.
- It's all plain text files that you own. Nothing's locked in; nothing leaves your machine."

3. A generic "what's next" paragraph

Give them one short, non-prescriptive paragraph. They now have a complete content brain, a foundation, and could, if they want, build their own content system on top of it. **Give no blueprint, no steps, and no mention of generating, scheduling, or posting mechanics.** Just point at the possibility and hand them the keys:

"That's the foundation done, a complete, organized brain of everything you've made and how you sound. What you build on top of it from here is entirely up to you. It's yours."

4. Congratulate them: specifically

Use the real counts. Make it land:

"This is a real milestone. You now have 214 videos, 96 articles, and 140 podcast episodes, all transcribed, tagged to your five pillars, and topped with your Voice DNA. Everything you've ever made, in one searchable place you own. Nicely done."

When this phase is done

1. Update `<vault>/content-brain/state.json` : set `phases.handoff` to `{ "done": true, "at": "<ISO timestamp now>", "notes": "<short summary>" }`, and mark the whole run complete (e.g. set a top-level `"complete": true` with the current timestamp).
2. **Do not load another phase, this is the end.**

[references/connector-playbook.md](#)

Connector Playbook

Purpose: a lookup table you consult to pick the best, safest way to pull in each kind of content. Match the source to a row, follow the recommended path, and do the "member action" note where the person has to do something themselves.

How to use this

- Detect the source type from what they told you in the interview (a URL, an app name, "it's a folder on my Mac").
- Prefer the least-effort path that gets clean text: existing captions and RSS beat re-downloading and re-transcribing.
- Copy nothing until they have approved it (Phase 2 assembles the plan, Phase 4 runs it).
- **Secrets stay with them.** If a path needs an API key or token, the member enters it into their own environment (a shell variable or the tool's own login). Never write a key or password into the vault, a note, a script, or `state.json`. If you need one, tell them exactly where to paste it and confirm it is set, never store the value yourself.

YouTube: channel, playlist, or single video

- **Detect:** a `youtube.com`, `youtu.be`, `@handle`, `/channel/`, `/playlist?list=`, or `/watch?v=` URL.
- **Path:** download audio + metadata + captions with yt-dlp; you turn the `.info.json` files into notes, then transcribe.
- **Tool:** `"${CLAUDE_PLUGIN_ROOT}"/scripts/ingest-youtube.sh <url> <vault>/wiki/sources/youtube`
- **Member action:** none for public content. For unlisted or members-only videos, they must give you the exact links (and yt-dlp may need their cookies, which they export themselves).

Podcast

- **Detect:** an Apple Podcasts or Spotify show page, or "my podcast."

- **Path:** find the RSS feed, then transcribe the audio.
 1. From an Apple Podcasts page: use WebFetch on the page and look for the feed, or ask the host platform.
 2. From Spotify (which hides the feed): ask the member where else the show is listed, or search the show name plus "RSS feed."
 3. Run `"${CLAUDE_PLUGIN_ROOT}"/scripts/ingest_rss.sh <feed-url> <vault>/wiki/sources/podcast` to save the feed and list audio enclosures.
 4. Run `"${CLAUDE_PLUGIN_ROOT}"/scripts/transcribe.sh <audio-file-or-dir> <vault>/wiki/sources/transcripts` on the enclosures.
- **Member action:** if you cannot find the feed, ask: "Can you paste the RSS link for your show? It's usually in your podcast host's settings, like Buzzsprout or Libsyn."

Newsletter: Substack / Beehiiv / Ghost / Mailchimp

- **Detect:** a newsletter domain, a `*.substack.com`, `*.beehiiv.com`, or "my newsletter."
- **Path:** most expose a public RSS feed.
 1. Try the feed first: Substack and Ghost are usually `<site>/feed`, Beehiiv `<site>/feed`. Run `"${CLAUDE_PLUGIN_ROOT}"/scripts/ingest_rss.sh <feed-url> <vault>/wiki/sources/articles`.
 2. If there is no feed, WebFetch the public archive page and each post into markdown.
 3. To recover past issues they SENT (not just public ones), the Gmail MCP can search their own outbox. Discover it via ToolSearch, then use `search_threads` / `get_thread`.
- **Member action:** if the archive is private, ask them to make it public briefly or to export from the platform. For Gmail, they must have Gmail connected as an MCP connector, only search their own mail, and only with their yes.

Blog / website articles

- **Detect:** a personal or company site with articles.
- **Path:**
 1. Try `<site>/sitemap.xml` (or `/sitemap_index.xml`) to enumerate every article URL.
 2. Also try an RSS feed (`/feed`, `/rss`, `/feed.xml`).
 3. WebFetch each article URL and save it as clean markdown into `<vault>/wiki/sources/articles`.
- **Member action:** none, if the site is public. For a members-only blog, they supply the login or export.

Google Docs / Google Drive

- **Detect:** a `docs.google.com` or `drive.google.com` link, or "my writing is in Google Docs."
- **Path:** use the Google Drive MCP. Discover it via ToolSearch, then `search_files` to find their docs, `get_file_metadata` to confirm, and `read_file_content` / `download_file_content` to pull the text.
- **Member action:** if Drive is not connected, say: "To reach your Google Docs I need you to connect Google Drive as a connector. Want me to wait while you do that?" If they would rather not connect it, ask them to export the docs (File > Download) to a local folder, then use `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder>`.

Notion

- **Detect:** a `notion.so` link or "my notes are in Notion."

- **Path (simplest):** ask them to export. In Notion: the `...` menu > Export > Markdown & CSV, for the page or whole workspace. Then `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <unzipped-export-folder> .`
- **Path (advanced):** the Notion API with their integration token, if they are comfortable creating one.
- **Member action:** run the export, or create a Notion integration and share the pages with it. If they use the token route, they paste it into their own environment, never into the vault.

Dropbox / iCloud / OneDrive

- **Detect:** files live in a cloud drive.
- **Path:** if the drive is synced to a local folder (most are, e.g. `~/Dropbox`, `~/Library/Mobile Documents/...` for iCloud, `~/OneDrive`), treat it as a local folder and use `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder> .` If it is online-only, they export or download a copy first.
- **Member action:** confirm the folder is fully downloaded (not just placeholders). For iCloud, files may need "Download Now" first. Consent required before any scan.

Local files on their Mac

- **Detect:** "it's in a folder," "on my desktop," "in my Documents."
- **Path:** `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder> .` It lists candidate files and copies nothing. Show them the list, let them pick, then ingest what they chose.
- **Member action:** they point you at the exact folder and say yes to the read-only scan first. Never scan a folder they have not named.

X / Twitter

- **Detect:** an `x.com` / `twitter.com` handle or "my tweets."
- **Path:** have them request their data archive, then ingest it.
 1. Member action: on X, Settings > "Your account" > "Download an archive of your data" (this can take a day or more to arrive).
 2. When the archive lands, it is a zip containing `data/tweets.js` (or a CSV). Point `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <unzipped-folder> .` at it, then read the tweets file and turn posts into notes.
- **Member action:** request the archive early since it is slow; flag it in Phase 2 as a "kick this off now" item.

LinkedIn

- **Detect:** a LinkedIn profile or "my LinkedIn posts."
- **Path:** have them request their export, then ingest it.
 1. Member action: LinkedIn > Settings & Privacy > "Data privacy" > "Get a copy of your data." Pick posts/articles. It arrives by email, often within minutes to a day.
 2. When it arrives, unzip and point `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder> .` at it; the posts come as CSV/HTML you turn into notes.
- **Member action:** request early; flag as a wait-for-export item in Phase 2.

Books / PDFs / ebooks

- **Detect:** a book they wrote, PDFs, `.epub` files, manuscripts.
- **Path:** these are local files. After `local_scan.sh` finds them, extract text by format:

- PDF: `pdftotext <file.pdf> <out.txt>` (from poppler), or macOS `textutil -convert txt <file.pdf>`.
- Word `.docx`: `textutil -convert txt <file.docx>` (built into macOS).
- EPUB: `ebook-convert <file.epub> <out.txt>` if Calibre is installed; if not, note the file for manual handling rather than guessing at its contents.
- **Member action:** they point you at the files and confirm they have the rights to them. If a format needs Calibre and it is missing, tell them and offer to move on: "Your EPUB needs a tool called Calibre to read. Want to install it, or should I skip these for now?"

Courses / webinars / recorded calls: Vimeo, Wistia, Zoom, Riverside

- **Detect:** a course platform, webinar replay, or recorded-call link.
- **Path:**
 1. If the URL is one yt-dlp supports (many Vimeo, Wistia, and public replay links are), use `"${CLAUDE_PLUGIN_ROOT}"/scripts/ingest_youtube.sh <url> <vault>/wiki/sources/...` (it drives yt-dlp, which handles far more than YouTube).
 2. If the platform blocks download or needs a login, have them download the recording from their own account, then treat it as a local file: `"${CLAUDE_PLUGIN_ROOT}"/scripts/local_scan.sh <folder>` then `transcribe.sh`.
- **Member action:** for gated content, they download their own copies (Zoom cloud recordings, Riverside exports, course downloads) into a folder and point you at it.

When nothing fits

If a source does not match any row, do not guess or fabricate. Tell the member plainly: "I don't have a clean automatic way to pull that one in. The reliable options are: export it to a folder, or give me a public link." Capture it as a member-action item and move on.